

version.gitlab.com Runbook

Overview

The **version.gitlab.com** application is the endpoint for self hosted GitLab instances to report their version to us (if that feature is enabled). The three primary functions of this app are:

1. Collect statistical information sent by self managed instances via http POST
2. Allow viewing and reporting on the statistical information collected
3. Serve **.svg** images to those instances indicating their upgrade status

This is an internally developed Rails app which is running on a GKE cluster, using an unmodified Auto DevOps deployment configuration. The production database is CloudSQL, and the staging/review databases currently run in pods provisioned by Auto DevOps.

The use of tools built into the GitLab product, in favor of technically better external solutions is intentional. The goal is to dogfood the operations and monitoring tools within the product, and use the discovered shortcomings to drive improvements to those areas. Building out tooling to work around these shortcomings is contrary to this goal.

Setup for On Call

- Read the README file for the GitLab Services Base project
- Note the location of the Metrics Dashboards
- Note the location of the CI Pipelines for the infrastructure components
- Note the location of the CI Pipelines for the application components

Workstation K8s Connection Setup

- ☐ Authenticate with **gcloud**

gcloud auth login

If you see warnings about permissions issues related to `~/.config/gcloud/*` check the permissions of this directory. Simply change it to your user if necessary: **sudo chown -R \$(whoami) ~/.config**

You'll be prompted to accept that you are using the **gcloud** on a shared computer and presented with a URL to continue logging in with, after which you'll be provided a code to pass into the command line to complete the process. By default, **gcloud** will configure your user within the same project configuration for which that **console** server resides.

- ☐ Get the credentials for production and staging:

```
gcloud container clusters get-credentials version-staging-gke --region us-east1 --project version-gcp
gcloud container clusters get-credentials version-prod-gke --region us-east1 --project version-gcp
```

Note that the hash after the project name may change without this documentation being updated. If in doubt, check the GCP console for the new hash.

This should add the appropriate context for `kubectl`, so the following should work and display the nodes running on the cluster:

```
❑ kubectl get nodes
```

Deployment

The application is deployed using Auto DevOps from the `version-gitlab-com` project. It uses a Review/Staging/Production scheme with no `.gitlab-ci.yml` file. If deployment problems are suspected, check for failed or incomplete jobs, and check the Environments page to make sure everything looks reasonable.

Note that the `gitlab-services` project is outside of the `gitlab-org` and `gitlab-com` namespaces. Everyone does not automatically have access to this project. If the above URL's result in 404 errors, chances are the user needs to be added to the project or group.

Project

The production deployment of the `version.gitlab.com` application is in the `gs-production` GCP project. The components to be aware of are:

- The Kubernetes cluster `gs-production-gke` and its node pool
- CloudSQL instance `cloudsql-7ea7` (the 4 character suffix is necessary for terraform and will change with future deployments)
- Load balancer (provisioned by the k8s ingress)
- Storage bucket `gs-production-db-backups` holds manual database exports. Do a manual export before any operations which touch cloudSQL, since it has been observed to lose data during operations which should be safe.

The review and staging deployments share the `version-staging-eae216` GCP project. The Kubernetes cluster is similar, but the databases are deployed as pods, so there is no CloudSQL instance

Database

The production database resides in a regional (`us-east1`) HA CloudSQL instance. The name will change when rebuilt. To retrieve the current name:

```
gcloud --project version-prod-674cf6 beta sql instances list
```

This instance is shared among the projects in the `gitlab-services` group. The database schema for the version application is `default`. The username and password can be found in the `DATABASE_URL` CI variable in the project settings.

Database backups are handled automatically by CloudSQL, and can be restored from the **Backups** tab of the CloudSQL instance. There are also occasional exports placed in the `gs-production-db-backups` bucket. These will not be as up to date, but they are easier to copy and move around.

Database access for developers

To grant access to database for developers (Example ARs: 16606, 13560), Cloud SQL **Viewer** and **Viewer** roles should be granted to requesting user on gcloud projects `version-prod-674cf6` and `version-staging-eae216` for production and staging respectively.

In some cases, specially when connecting user has IPv6 address, use `beta` switch with gcloud command as follows:

```
gcloud --project version-prod-674cf6 beta sql connect cloudsql-7ea7 -u default
```

Password for `default` user can be found from `DATABASE_URL` CI variable in project settings as mentioned above

Terraform

This GCP project and the infrastructure components in it are managed by the `gitlab-com/gl-infra/gitlab-services` project. Any infrastructure changes to the environment or K8s cluster should be made as an MR there. Changes will be applied automatically via CI jobs when the MR is merged. `version-prod` and `version-staging` are represented as Environments in that project.

This workflow is different from other areas of the infrastructure. `services-base` uses the GitLab Flow Workflow. There is currently no manual step between `terraform plan` and `terraform apply`. The assumption is that an ephemeral environment in a review stage doesn't need this, and for a production environment any change must have successful pipelines in both the review stage and the master merge before they can be applied to the production branch. We may revisit this as these environments mature.

Monitoring

Monitoring is handled from within the GitLab application, using the built in monitoring functionality. This is done to dogfood the built in monitoring tools. Any shortcomings should be pointed out using GitLab Product issues and labelled for the Monitor team. The Prometheus instance used is deployed via the Kubernetes Integration page.

The issue discussing setup of the monitoring dashboards is <https://gitlab.com/gitlab-services/version-gitlab-com/issues/185>

Checking the Ingress

Switch contexts to the `gs-production-gke` cluster in the `gs-production` namespace.

Make sure there is at least one ingress controller pod, and that it hasn't been restarting. Note the age and restart count in the below example output.

```
% kubectl get pods -n gitlab-managed-apps -l app=nginx-ingress
```

NAME	READY	STATUS	RESTARTS	AGE
ingress-nginx-ingress-controller-85ff56cfdd-cjd9b	1/1	Running	0	24h
ingress-nginx-ingress-controller-85ff56cfdd-fmqnh	1/1	Running	0	24h
ingress-nginx-ingress-controller-85ff56cfdd-tg77w	1/1	Running	0	46h
ingress-nginx-ingress-default-backend-76d9f87474-xm66d	1/1	Running	0	46h

Check for Events:

```
kubectl describe deployment -n gitlab-managed-apps ingress-nginx-ingress-controller
```

The bottom of this output will show health check failures, pod migrations and restarts, and other events which might effect availability of the ingress. **Events:** `<none>` means the problem is probably elsewhere.

After 1 hour, these events are removed from the output, so historical information can be found in the stackdriver logs

Rebuilding or upgrading the ingress

Currently, the integration does not have a way to upgrade components. To upgrade the ingress controller:

1. Submit a production change issue to schedule a maintenance window
2. Go to the Kubernetes integration page, and uninstall the ingress controller
3. Once it finishes, click the install button
4. The IP address will change. Take this new IP address and replace the existing one in the DNS for the wildcard entry on that page, as well as any site specific entries (`version.gitlab.com` in this case).

Certificates

Certificates are managed by the `cert-manager` pod installed via the Kubernetes Integration page. This will handle automatic renewals. All of this only works if all DNS entries named in the certificate point to the ingress IP.

DNS

DNS is hosted in route53, and is managed via terraform in the gitlab-com-infra repository

Resources

Switch contexts to the `gs-production-gke` cluster in the `gs-production` namespace.

The overall usage can be checked like this:

```
$ kubectl top nodes
```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
gke-gs-production-gke-node-pool-0-08bfc75b-v8dk	132m	1%	3183Mi	11%
gke-gs-production-gke-node-pool-0-a6855491-hrx5	125m	1%	2534Mi	9%
gke-gs-production-gke-node-pool-0-e198996d-jwk0	178m	2%	1705Mi	6%

Pods can be checked like this:

```
kubectl top pods --all-namespaces
```

NAMESPACE	NAME
gitlab-managed-apps	certmanager-cainjector-7f7bbbdd96-2gpl
gitlab-managed-apps	certmanager-cert-manager-596ffbc84-k5r99
gitlab-managed-apps	certmanager-webhook-79649b6846-r9v5v
gitlab-managed-apps	ingress-nginx-ingress-controller-85ff56cfdd-cjd9b
gitlab-managed-apps	ingress-nginx-ingress-controller-85ff56cfdd-fmqnh
gitlab-managed-apps	ingress-nginx-ingress-controller-85ff56cfdd-tg77w
gitlab-managed-apps	ingress-nginx-ingress-default-backend-76d9f87474-xm
gitlab-managed-apps	prometheus-kube-state-metrics-5d5958bc-xp9rw
gitlab-managed-apps	prometheus-prometheus-server-5c476cc89-nr6kl
gitlab-managed-apps	runner-gitlab-runner-795f7d855c-sjsnk
gitlab-managed-apps	tiller-deploy-5c85978967-c9lpx
kube-system	event-exporter-v0.2.5-7df89f4b8f-zj2fn
kube-system	fluentd-gcp-scaler-54ccb89d5-f7kzr
kube-system	fluentd-gcp-v3.1.1-ktq4k
kube-system	fluentd-gcp-v3.1.1-qvl4v
kube-system	fluentd-gcp-v3.1.1-z979w
kube-system	heapster-554bd74c87-tjdpn
kube-system	kube-dns-5877696fb4-48xp7
kube-system	kube-dns-5877696fb4-r8rp4
kube-system	kube-dns-autoscaler-85f8bdb54-52zgr
kube-system	kube-proxy-gke-gs-production-gke-node-pool-0-08bfc75b
kube-system	kube-proxy-gke-gs-production-gke-node-pool-0-a6855491
kube-system	kube-proxy-gke-gs-production-gke-node-pool-0-e198996d
kube-system	17-default-backend-fd59995cd-8sntz
kube-system	metrics-server-v0.3.1-57c75779f-z8whn
kube-system	prometheus-to-sd-gm9zz

kube-system	prometheus-to-sd-s8p6w
kube-system	prometheus-to-sd-zx4t7
kube-system	stackdriver-metadata-agent-cluster-level-8597c4d686-
kube-system	tiller-deploy-5f4fc5bcc6-zzts2
version-gitlab-com-6491770-production	production-65577f7bc4-7g4dx
version-gitlab-com-6491770-production	production-65577f7bc4-bqqnj
version-gitlab-com-6491770-production	production-65577f7bc4-dbm7z
version-gitlab-com-6491770-production	production-65577f7bc4-dxrhv
version-gitlab-com-6491770-production	production-65577f7bc4-fp9tp
version-gitlab-com-6491770-production	production-65577f7bc4-fs7v6

Alerting

Currently, the only alerting is the pingdom blackbox alerts. This is the same as what was set up in the previous AWS environment, but probably needs to be improved. The preference is to use built in GitLab functionality where possible.

There is work to improve the current alerting mechanism inside of the GitLab product. This work can be followed here: <https://gitlab.com/gitlab-org/gitlab/issues/30832>